

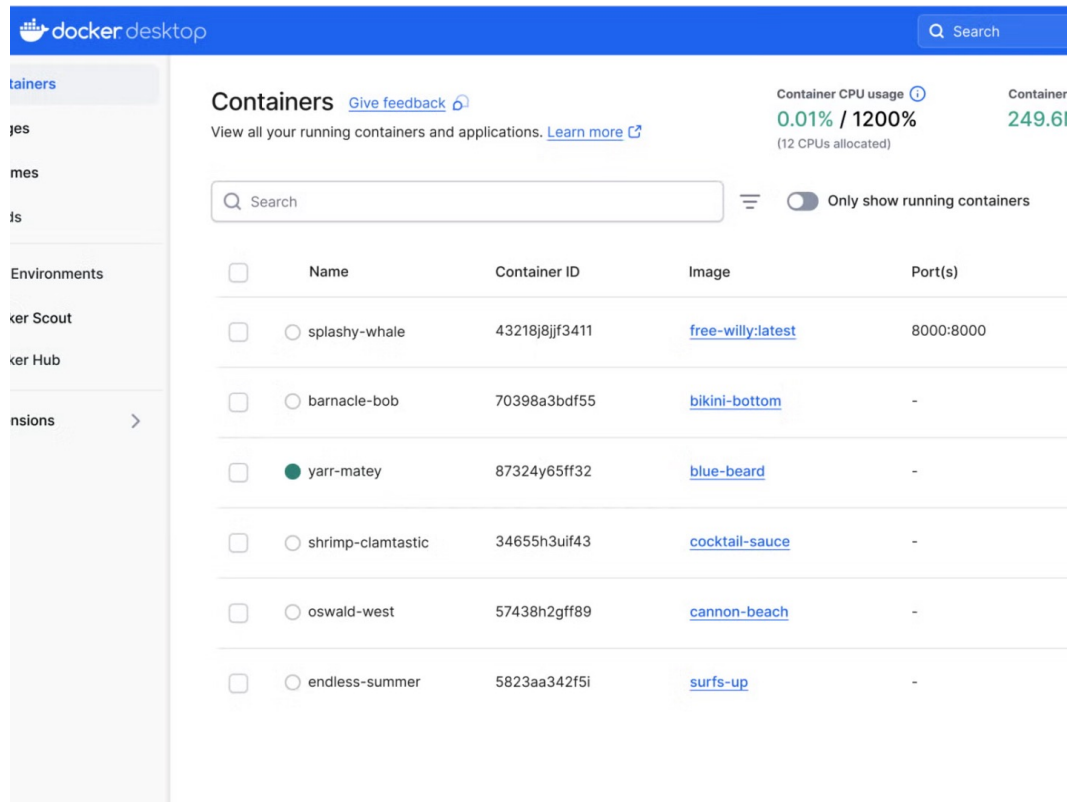
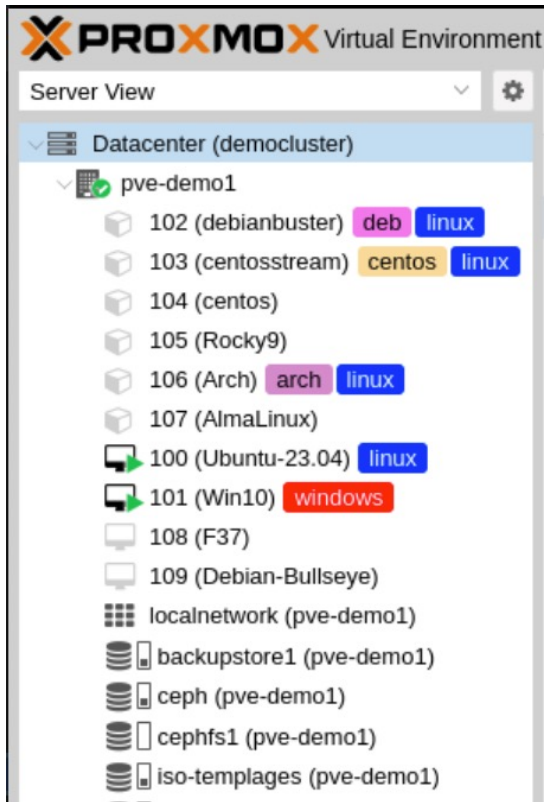
Containers

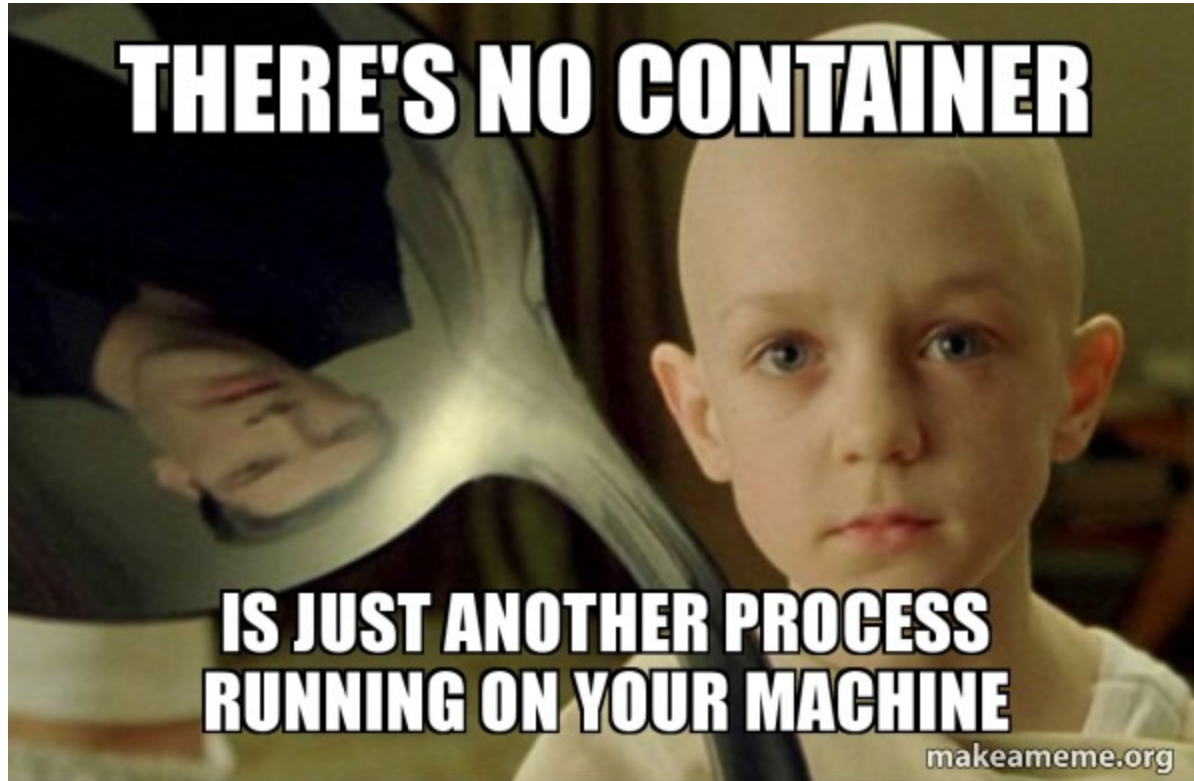
A look under the hood



What is a container?

What does it look like from the host perspective?





```

➔ ps auxf | tail -n 200 | head -n 50
100000 542181 0.0 0.0 8040 1616 pts/0 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 linux
100000 542183 0.0 0.0 8040 1624 pts/1 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100000 542185 0.0 0.0 8040 1592 pts/2 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100000 542186 0.0 0.0 15592 7320 ? Ss May24 0:00 \_ sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
100000 542280 0.0 0.0 83244 2448 ? Ss May24 0:00 \_ nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
100033 542283 0.0 0.0 84096 7424 ? S May24 0:40 | \_ nginx: worker process
100033 542284 0.0 0.0 87196 10556 ? S May24 0:57 | \_ nginx: worker process
100033 542285 0.0 0.0 84256 7480 ? S May24 2:00 | \_ nginx: worker process
100033 542286 0.0 0.0 84184 7512 ? S May24 1:07 | \_ nginx: worker process
100000 542379 0.0 0.0 42676 2364 ? Ss May24 0:00 \_ /usr/lib/postfix/sbin/master -w
100101 542381 0.0 0.0 42740 4384 ? S May24 0:00 \_ qmgr -l -t unix -u
100101 1113257 0.0 0.0 42696 4312 ? S 17:45 0:00 \_ pickup -l -t unix -u -c
root 541903 0.0 0.0 6652 3892 ? Ss May24 0:14 /usr/bin/lxc-start -F -n 230
100000 542049 0.0 0.0 21408 10396 ? Ss May24 0:01 \_ /sbin/init
100000 542536 0.0 0.0 34368 11808 ? Ss May24 0:01 \_ /usr/lib/systemd/systemd-journald
100105 542607 0.0 0.0 19008 7156 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-networkd
100106 542728 0.0 0.0 21472 9328 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-resolved
100000 542737 0.0 0.0 307600 5452 ? Ssl May24 0:02 \_ /usr/libexec/accounts-daemon
100000 542738 0.0 0.0 3816 1868 ? Ss May24 0:00 \_ /usr/sbin/cron -f -P
100100 542739 0.0 0.0 9600 3788 ? Ss May24 0:00 \_ atdbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-a
100110 542742 0.2 0.0 981104 60956 ? Ssl May24 9:22 \_ /usr/bin/mongod --unixSocketPrefix=/run/mongoddb --config /etc/mongoddb.conf
100000 542743 0.0 0.0 29376 17336 ? Ss May24 0:00 \_ /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
100000 542746 0.0 0.0 17528 6280 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-logind
100000 542789 0.0 0.0 2740 1344 pts/0 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 linux
100000 542792 0.0 0.0 2740 1444 pts/1 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100000 542793 0.0 0.0 2740 1396 pts/2 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100998 542804 0.0 0.0 308172 5764 ? Ssl May24 0:03 \_ /usr/lib/polkit-1/polkitd --no-debug
100101 542869 0.0 0.0 152880 4004 ? Ssl May24 0:00 \_ /usr/sbin/rsyslogd -n -iNONE
100111 542882 0.2 1.7 4956268 1138528 ? Ssl May24 9:44 \_ /usr/bin/java -Dfile.encoding=UTF-8 -Djava.awt.headless=true -Dapple.awt.
rorFile=/usr/lib/unifi/logs/unifi_crash.log -Xlog:gc:logs/gc.log:time:filecount=2,filesize=5M --add-opens java.base/java.lang=ALL-UNNAMED --add-open
NNAMED -jar /usr/lib/unifi/lib/ace.jar start
100111 543101 0.2 0.2 1140116 186616 ? Sl May24 11:38 | \_ bin/mongod --dbpath /usr/lib/unifi/data/db --port 27117 --bind_ip 127
rEngineConfigString=cache_size=256M
100000 543004 0.0 0.0 42864 2040 ? Ss May24 0:00 \_ /usr/lib/postfix/sbin/master -w
100102 543006 0.0 0.0 43348 4860 ? S May24 0:00 \_ qmgr -l -t unix -u
100102 1113417 0.0 0.0 43308 4752 ? S 17:45 0:00 \_ pickup -l -t unix -u -c
root 543660 0.0 0.0 2572 888 ? Ss May24 0:00 /usr/bin/dtach -A /var/run/dtach/vzctlconsole230 -r winch -z lxc-console -n 230 -e
root 543661 0.0 0.0 6644 3556 pts/6 Ss+ May24 0:00 \_ lxc-console -n 230 -e -1
root 574516 0.0 0.0 2572 880 ? Ss May24 0:00 /usr/bin/dtach -A /var/run/dtach/vzctlconsole140 -r winch -z lxc-console -n 140 -e
root 574517 0.0 0.0 6644 3484 pts/8 Ss+ May24 0:00 \_ lxc-console -n 140 -e -1
root 577181 0.0 0.0 2572 816 ? Ss May24 0:00 /usr/bin/dtach -A /var/run/dtach/vzctlconsole220 -r winch -z lxc-console -n 220 -e
root 577182 0.0 0.0 6644 3520 pts/9 Ss+ May24 0:00 \_ lxc-console -n 220 -e -1
root 709328 0.0 0.0 6652 3928 ? Ss May24 0:14 /usr/bin/lxc-start -F -n 130
100000 709373 0.0 0.0 22120 11392 ? Ss May24 0:09 \_ /sbin/init
100000 709540 0.0 0.1 165736 107944 ? Ss May24 0:16 \_ /usr/lib/systemd/systemd-journald
100101 709601 0.0 0.0 19008 7524 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-networkd
100102 709711 0.0 0.0 21472 10332 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-resolved
100000 709732 0.0 0.0 310724 5796 ? Ssl May24 0:03 \_ /usr/libexec/accounts-daemon
100109 709735 0.0 0.0 156676 15960 ? Ss May24 0:01 \_ /usr/bin/freshclam -d --foreground=true
100000 709736 0.0 0.0 6880 2144 ? Ss May24 0:02 \_ /usr/sbin/cron -f -P
100103 709737 0.0 0.0 9684 4136 ? Ss May24 0:04 \_ atdbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-a
100000 709739 0.0 0.0 8608 3720 ? Ss May24 0:05 \_ /usr/sbin/dovecot -F
102000 709806 0.0 0.0 13080 6964 ? S May24 0:00 | \_ dovecot/lmtp -L

```

```

→ ps auxf | tail -n 200 | head -n 50
100000 542181 0.0 0.0 8040 1616 pts/0 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 linux
100000 542183 0.0 0.0 8040 1624 pts/1 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100000 542185 0.0 0.0 8040 1592 pts/2 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100000 542186 0.0 0.0 15592 7320 ? Ss May24 0:00 \_ sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
100000 542280 0.0 0.0 83244 2448 ? Ss May24 0:00 \_ nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
100033 542283 0.0 0.0 84096 7424 ? S May24 0:40 | \_ nginx: worker process
100033 542284 0.0 0.0 87196 10556 ? S May24 0:50 | \_ nginx: worker process
100033 542285 0.0 0.0 84256 7480 ? S May24 2:00 | \_ nginx: worker process
100033 542286 0.0 0.0 84184 7512 ? S May24 1:07 | \_ nginx: worker process
100000 542379 0.0 0.0 42676 2364 ? Ss May24 0:00 \_ /usr/lib/postfix/sbin/master -w
100101 542381 0.0 0.0 42740 4384 ? S May24 0:00 \_ qmgr -l -t unix -u
100101 1113257 0.0 0.0 42696 4312 ? S 17:45 0:00 \_ pickup -l -t unix -u -c
root 541903 0.0 0.0 6652 3892 ? Ss May24 0:14 /usr/bin/lxc-start -F -n 230
100000 542049 0.0 0.0 21408 10396 ? Ss May24 0:01 \_ /sbin/init
100000 542536 0.0 0.0 34368 11808 ? Ss May24 0:01 \_ /usr/lib/systemd/systemd-journald
100105 542607 0.0 0.0 19008 7156 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-networkd
100106 542728 0.0 0.0 21472 9328 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-resolved
100000 542737 0.0 0.0 307600 5452 ? Ssl May24 0:02 \_ /usr/libexec/accounts-daemon
100000 542738 0.0 0.0 3816 1868 ? Ss May24 0:00 \_ /usr/sbin/cron -f -P
100100 542739 0.0 0.0 9600 3788 ? Ss May24 0:00 \_ dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-a
100110 542742 0.2 0.0 981104 60956 ? Ssl May24 9:22 \_ /usr/bin/mongod --unixSocketPrefix=/run/mongod --config /etc/mongod.conf
100000 542743 0.0 0.0 29376 17336 ? Ss May24 0:00 \_ /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
100000 542746 0.0 0.0 17528 6280 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-logind
100000 542789 0.0 0.0 2740 1344 pts/0 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 linux
100000 542792 0.0 0.0 2740 1444 pts/1 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100000 542793 0.0 0.0 2740 1396 pts/2 Ss+ May24 0:00 \_ /sbin/agetty -o -p -- \u --noclear - linux
100998 542804 0.0 0.0 308172 5764 ? Ssl May24 0:03 \_ /usr/lib/polkit-1/polkitd --no-debug
100101 542869 0.0 0.0 152880 4004 ? Ssl May24 0:00 \_ /usr/sbin/rsyslogd -n -iNONE
100111 542882 0.2 1.7 4956268 1138528 ? Ssl May24 9:44 \_ /usr/bin/java -Dfile.encoding=UTF-8 -Djava.awt.headless=true -Dapple.awt.
rorFile=/usr/lib/unifi/logs/unifi_crash.log -Xlog:gc:logs/gc.log:time,filecount=2,filesize=5M --add-opens java.base/java.lang=ALL-UNNAMED --add-open
NNAMED -jar /usr/lib/unifi/lib/ace.jar start
100111 543101 0.2 0.2 1140116 186616 ? Sl May24 11:38 | \_ bin/mongod --dbpath /usr/lib/unifi/data/db --port 27117 --bind_ip 127
EngineConfigStrings:cache_size=256M
100000 543004 0.0 0.0 42864 2040 ? Ss May24 0:00 \_ /usr/lib/postfix/sbin/master -w
100102 543006 0.0 0.0 43348 4860 ? S May24 0:00 \_ qmgr -l -t unix -u
100102 1113417 0.0 0.0 43308 4752 ? S 17:45 0:00 \_ pickup -l -t unix -u -c
root 543660 0.0 0.0 2572 888 ? Ss May24 0:00 /usr/bin/dtatch -A /var/run/dtatch/vzctlconsole230 -r winch -z lxc-console -n 230 -
root 543661 0.0 0.0 6644 3556 pts/6 Ss+ May24 0:00 \_ lxc-console -n 230 -e -1
root 574516 0.0 0.0 2572 880 ? Ss May24 0:00 /usr/bin/dtatch -A /var/run/dtatch/vzctlconsole140 -r winch -z lxc-console -n 140 -
root 574517 0.0 0.0 6644 3484 pts/8 Ss+ May24 0:00 \_ lxc-console -n 140 -e -1
root 577181 0.0 0.0 2572 816 ? Ss May24 0:00 /usr/bin/dtatch -A /var/run/dtatch/vzctlconsole220 -r winch -z lxc-console -n 220 -
root 577182 0.0 0.0 6644 3520 pts/9 Ss+ May24 0:00 \_ lxc-console -n 220 -e -1
root 709328 0.0 0.0 6652 3928 ? Ss May24 0:14 /usr/bin/lxc-start -F -n 130
100000 709373 0.0 0.0 22120 11392 ? Ss May24 0:09 \_ /sbin/init
100000 709540 0.0 0.1 165736 107944 ? Ss May24 0:16 \_ /usr/lib/systemd/systemd-journald
100101 709601 0.0 0.0 19008 7524 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-networkd
100102 709711 0.0 0.0 21472 10332 ? Ss May24 0:00 \_ /usr/lib/systemd/systemd-resolved
100000 709732 0.0 0.0 310724 5796 ? Ssl May24 0:03 \_ /usr/libexec/accounts-daemon
100109 709735 0.0 0.0 156676 15960 ? Ss May24 0:01 \_ /usr/bin/freshclam -d --foreground=true
100000 709736 0.0 0.0 6880 2144 ? Ss May24 0:02 \_ /usr/sbin/cron -f -P
100103 709737 0.0 0.0 9684 4136 ? Ss May24 0:04 \_ dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-a
100000 709739 0.0 0.0 8608 3720 ? Ss May24 0:05 \_ /usr/sbin/dovecot -F
102000 709806 0.0 0.0 13080 6964 ? S May24 0:00 | \_ dovecot/lmtp -L

```

What is a container?

A container is a regular process on the host system to which certain:

- **resource consumption limits,**
- **resource visibility limits,**
- **and security policies**
are applied.

„limitation“

„security“

„isolation“

Who enforces the limits?

Docker? OpenVZ? LXC? Proxmox?

No. The kernel does.

Features to enable containers:

- **Linux kernel primitives**
- **Linux kernel access controls**

Kernel primitives enabling containers

Linux uses two key technologies to achieve **limiting** and **isolating** processes:

- **control groups (cgroups)**
 - “resource management / limiting”
- **namespaces (ns)**
 - “process isolation / virtual resources”

Both of these technologies are part of the modern Linux kernel, while their userland tooling is typically installed separately. LXC, LXD, Docker, OpenVZ, etc. only provide very-high-level abstraction / tools to efficiently configure and manage containers, ran by OS and enabled by the kernel’s features.

Kernel primitives enabling containers

Additionally, **modern filesystems and volume managers** (snapshots, cow, overlays, ...) enable efficient:

- **root filesystem (rootfs)**
 - “file(s)/image for starting the process and setting up the environment”
 - +pivot_root() or other transition mechanism

Control groups (cgroups)

Introduced by Google in 2007, enter mainline in 2008 => LXC

Cgroups enable **resource management**:

- allocate, prioritize, deny, manage, and monitor
- CPU time, system memory, network bandwidth, etc.

Core idea: group processes (tasks) and apply group resource limits

Use in LXC / Docker: container resource limits

- prevent monopolizing host system resources
- performance optimization in shared environments

Control groups (cgroups)

Under the hood, cgroups make use of **kernel resource controllers**:

- **io** (set input/output limits/shares on resources)
- **memory** (set memory use limits and reporting)
- **pids** (limit number of processes and their children)
- **rdma** (set Remote DMA / InfiniBand resource limits)
- **cpu** (adjust scheduler parameters e.g., Completely Fair Scheduler)
- **cpuset** (restrict available cpu and memory nodes)
- **perf_event** (performance monitoring and reporting)

Under the hood => cpu controller integrates with linux scheduler (cpu time)
memory controller integrates with kernel memory manager

Control groups (cgroups)

We don't need Docker, LXC, etc. to create and manage cgroups:

- interaction via filesystem interface **/sys/fs/cgroup/**
 - in UNIX "everything is a file", pseudo-filesystem
- using userland tools e.g. **cgroup-tools, cgutils, libcgroup**

```
ls -lh /sys/fs/cgroup
```

```
-r--r--r-- 1 root root 0 Dec 12 16:38 cgroup.controllers
-rw-r--r-- 1 root root 0 Dec 12 16:38 cgroup.max.depth
-rw-r--r-- 1 root root 0 Dec 12 16:38 cgroup.max.descendants
-rw-r--r-- 1 root root 0 Dec 12 16:38 cgroup.pressure
-rw-r--r-- 1 root root 0 Dec 12 16:38 cgroup.procs
-r--r--r-- 1 root root 0 Dec 12 16:38 cgroup.stat
-rw-r--r-- 1 root root 0 Dec 12 16:38 cgroup.subtree_control
-rw-r--r-- 1 root root 0 Dec 12 16:38 cgroup.threads
```

Control groups (cgroups)

We don't need LXC, Docker, etc. to create and manage cgroups:

We can now view or modify maximum allowed cgroup memory using standard file operations:

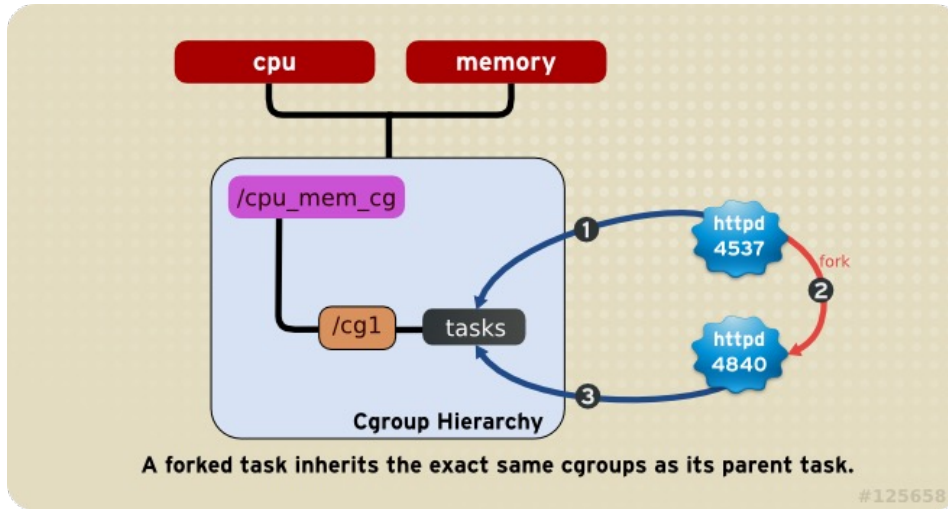
```
# get max memory
cat /sys/fs/cgroup/mygroup/memory.max # result in bytes
# add PID 1234 to group
echo 1234 >> /sys/fs/cgroup/mygroup/cgroup.procs
```

Linux uses inotify to monitor for fs changes and apply resource controller policies in real-time.

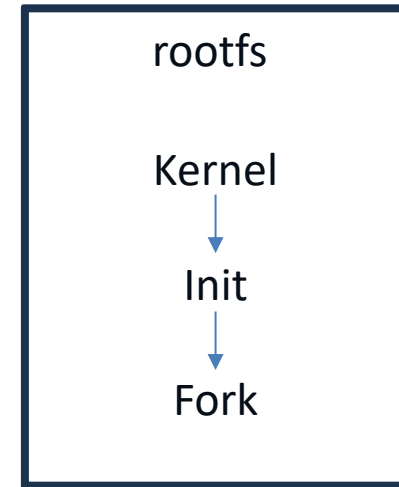
Control groups (cgroups)

Towards containers

cgroups hierarchies follow a set of rules – child task automatically inherits cgroup membership of its parent, but can be moved



Container



Namespaces

enable **resource and process isolation** and allow for the partitioning of various aspects of the operating system, so that each set of processes sees its own isolated instance of the global resource.

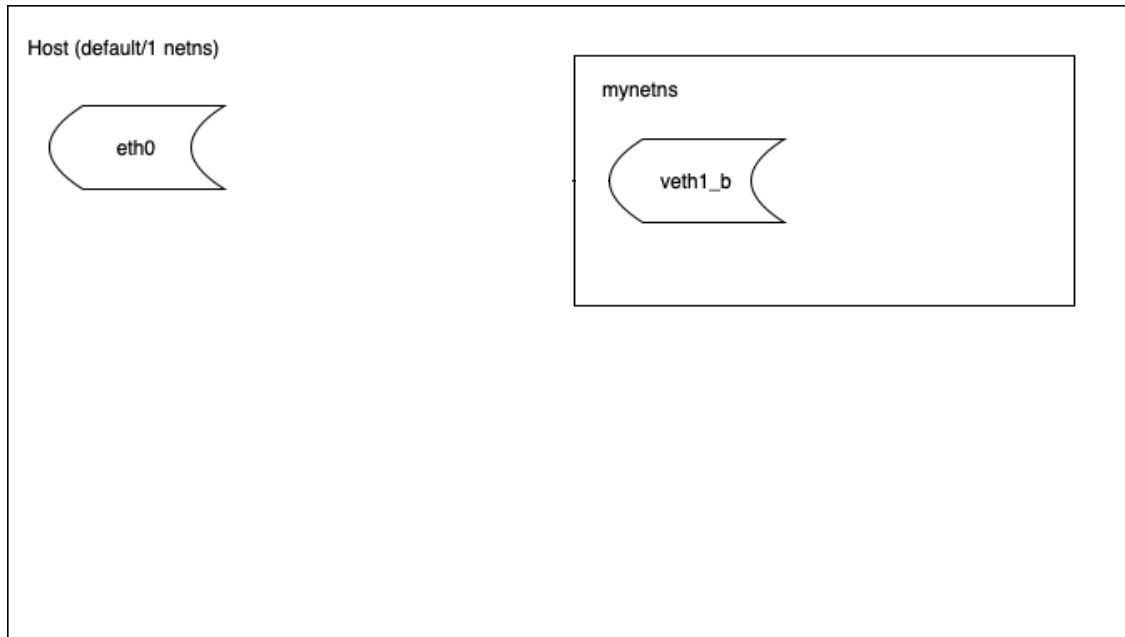
Namespaces provide **virtual environments for processes**, isolating them from other processes and resources. Environments / namespace types:

- **Mount:** CLONE_NEWNS - Mount points
- **Network:** CLONE_NEWNET - Network devices, stacks, firewall, routing tables, etc.
- **User:** CLONE_NEWUSER - User and group IDs
- **PID:** CLONE_NEWPID - Process IDs
- **UTS:** CLONE_NEWUTS - Hostname and NIS domain name
- **IPC:** CLONE_NEWIPC - System V IPC, POSIX message queues
- **Cgroup:** CLONE_NEWCGROUP - Cgroup root directory

Namespaces

Towards containers

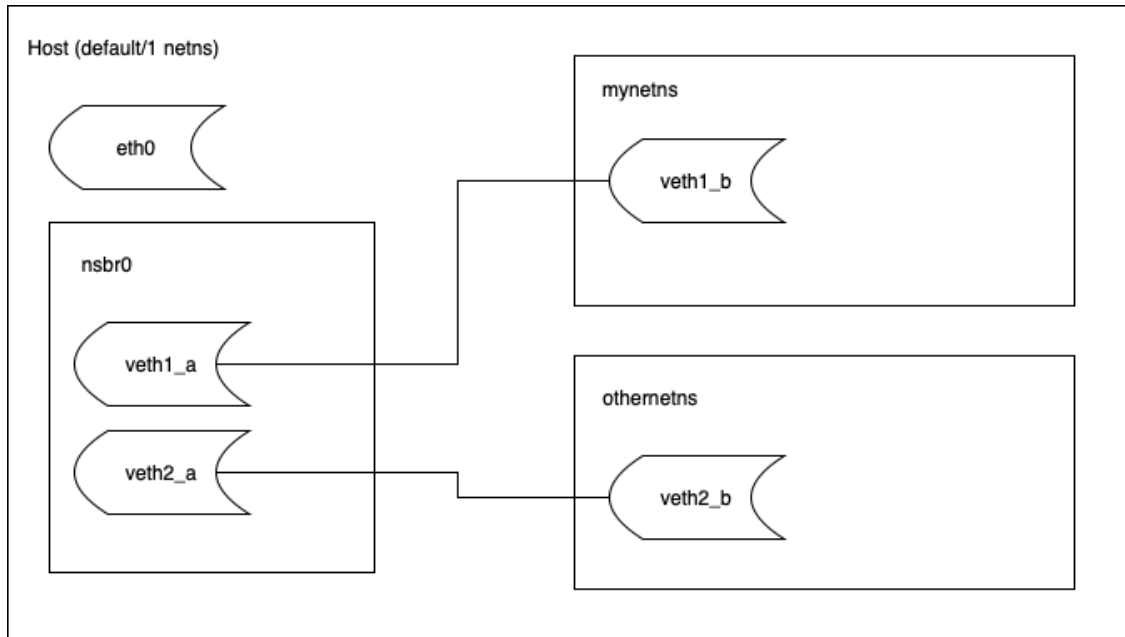
Network namespace



Namespaces

Towards containers

Network namespace



Namespaces

Towards containers

Network namespace

```
sudo ip netns add mynetns # add a network namespace
```

.... configure new IP, routes, etc. inside mynetns

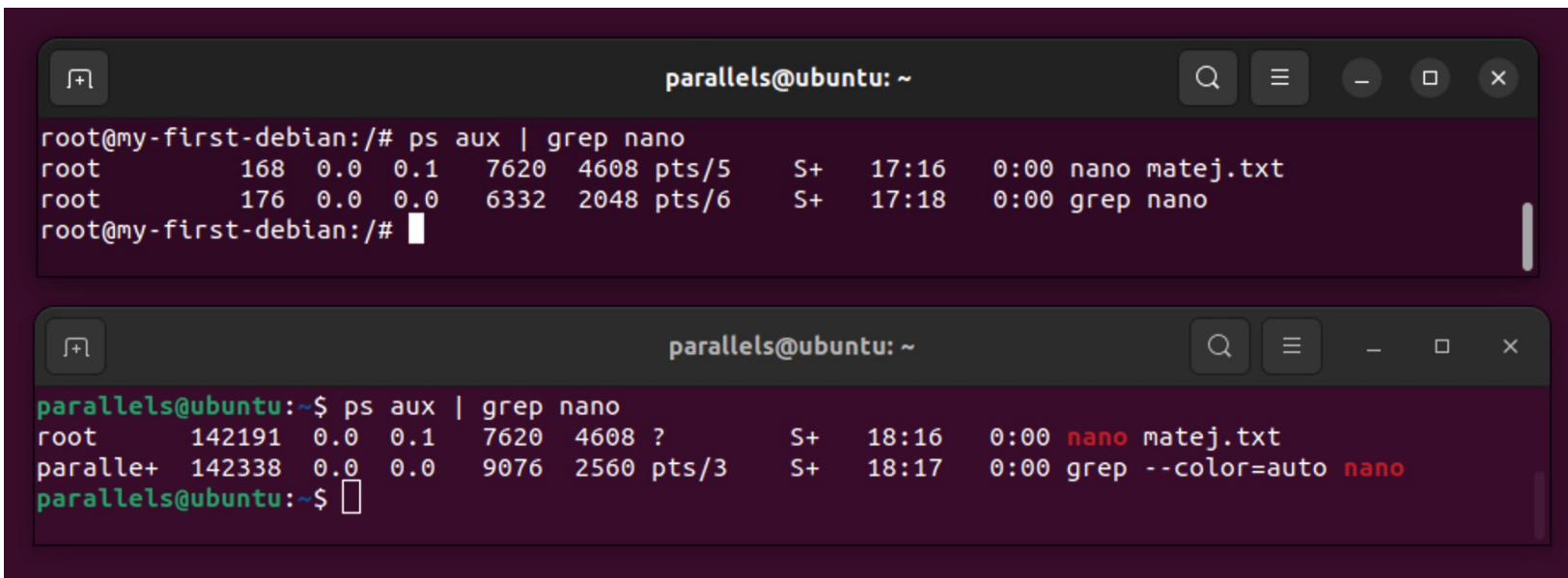
```
sudo ip netns exec mynetns bash  
ping 1.1.1.1
```

Namespaces

Towards containers

Mount namespace (chroot / pivot_root)

PID Namespace (remapping process IDs)



The image shows two terminal windows side-by-side, both titled 'parallels@ubuntu: ~'. The top window shows a process listing from inside a container named 'my-first-debian'. The bottom window shows a process listing from the host system 'parallels@ubuntu'.

```
root@my-first-debian:/# ps aux | grep nano
root      168  0.0  0.1   7620  4608 pts/5    S+   17:16   0:00 nano matej.txt
root      176  0.0  0.0   6332  2048 pts/6    S+   17:18   0:00 grep nano
root@my-first-debian:/#
```

```
parallels@ubuntu:~$ ps aux | grep nano
root      142191  0.0  0.1   7620  4608 ?        S+   18:16   0:00 nano matej.txt
paralle+  142338  0.0  0.0   9076  2560 pts/3    S+   18:17   0:00 grep --color=auto nano
parallels@ubuntu:~$
```

What about the security?

- cgroups can limit resource exhaustion (DoS)
- namespaces can isolate resources (fs, dev, ...)

... but the kernel is still shared! These are not security features!

- containers/processes can still access kernel interfaces
- they can make system calls like any other process
- if they run as root (even in namespace and cgroup) they are root on the host

Securing containers

We implement security mechanisms to further secure the confined processes.

Traditional Linux security controls (DAC) are composed from a variety of mechanisms:

- **users and groups**
- **FS permissions**
- **POSIX ACLs**
- **setuid/setgid bits**
- **sudoers ...**

Root can do everything!

The problem: we may need to run some containers as **root** (e.g., **to access a host device, to bind to ports < 1024, to perform certain syscalls**) but want to **limit everything else** they can do.

The solution: **we need to limit what root can do per process!**

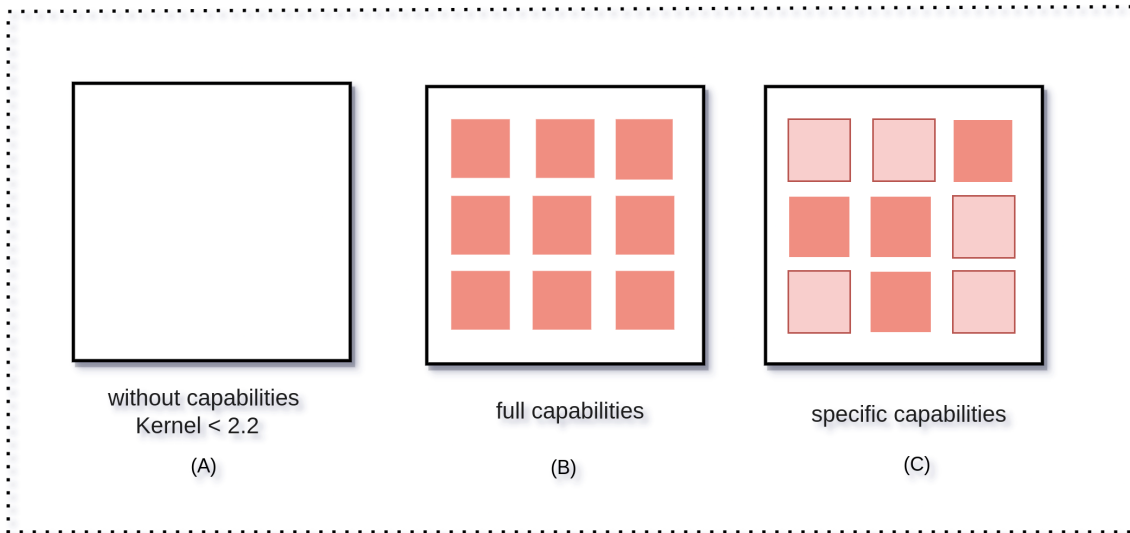
Kernel access controls

Besides **cgroups**, **namespaces**, and **rootfs**, dedicated **kernel access controls** enable workload security:

- kernel security mechanisms:
 - **linux capabilities** (subdividing root permissions)
 - **seccomp** (limiting system calls)
 - **linux security modules** (limiting root permissions)

Linux capabilities

TLDR: Subdivide root permissions into 41 individually assignable capabilities.



A process running as root can now only perform part of the root actions or vice versa a standard user can use a capability without becoming root.

Example:

- + add `CAP_SYS_RAWIO` to access IO devices
- remove `CAP_NET_BIND` to prevent network port binding

Ping!

seccomp

Capabilities cut down *which kernel features* a container can call.

Seccomp restricts *which syscalls* it can touch.

A Linux kernel has 350+ syscalls. Most workloads need 40–60.

- seccomp-bpf lets you write a small BPF program that decides "allow / deny" per syscall
- Docker ships a default seccomp profile that blocks 50 dangerous syscalls
- The default profile is reasonable but not minimal

Example policy:

```
deny_syscall(ctx, SCMP_SYS(reboot), "reboot");  
deny_syscall(ctx, SCMP_SYS(mount), "mount");
```

LSM / MAC

Linux Security Modules further restrict *which files* it can touch. **Mandatory Access Control (MAC)** is the kernel-enforced layer that says "no, you can't do that" *regardless of file permissions or UID*.

Example: **AppArmor**: "this process may read `/etc/*` but not `/root/*`".
It also handles capabilities, IPC, etc.

- Default on Debian, Ubuntu, SUSE.
- Docker ships a default profile (`docker-default`) for every container.

Example: block process from
accessing host kernel interfaces
which are mounted in guest!

```
deny /proc/sys/kernel/** w,  
deny /root/alpine-rootfs/proc/sys/kernel/** w,
```

More on security later on

- no amount of hardening the guests will fix a misconfigured host
- „privileged“ means „no fence“ and differs between technologies (e.g., Docker, LXC)
- defaults are not safe, you need defence in depth:

Network — bind to 127.0.0.1, use internal networks, run a reverse proxy in front

Image — minimal base, no shell, USER non-root, no unnecessary binaries

Capabilities — --cap-drop=ALL, then add only what you need

seccomp — keep the default profile or write a tighter one

LSM — AppArmor or SELinux profile applied

User namespace / rootless — container root is host nobody

Read-only rootfs + no-new-privileges + tmpfs for /tmp

Creating containers

We now combine the described mechanisms to start a new container:

1. Creating isolated namespaces.
 - e.g., isolating the filesystem similar to chroot, creating new netns with independent networking stack, etc.
2. Applying resource limits with cgroups.
 - e.g., setting cpu and memory limits, throttling network interfaces, etc.
3. Implementing security measures with capabilities.
 - e.g., allow creating sockets, define container "privilege" level (more on that later)
4. Starting an init process within the container.
 - init assumes PID 1 inside the container PID ns and starts userspace environment, the init system may differ from the one used by the host (e.g., sysvinit vs systemd)

■ That's it!

